

The world's first Ajax server

[Home](#)
[Quick Start](#)
[Guide](#)
[Tutorials](#)
[API](#)
[Screencasts](#)
[Forums](#)
[Contribute](#)
[Download](#)

Jaxer WikiLite Walkthrough

This sample ships with code updated for Jaxer 1.0 and Aptana Studio 1.2 and is a good source of information but the tutorial has not yet been revised to match.

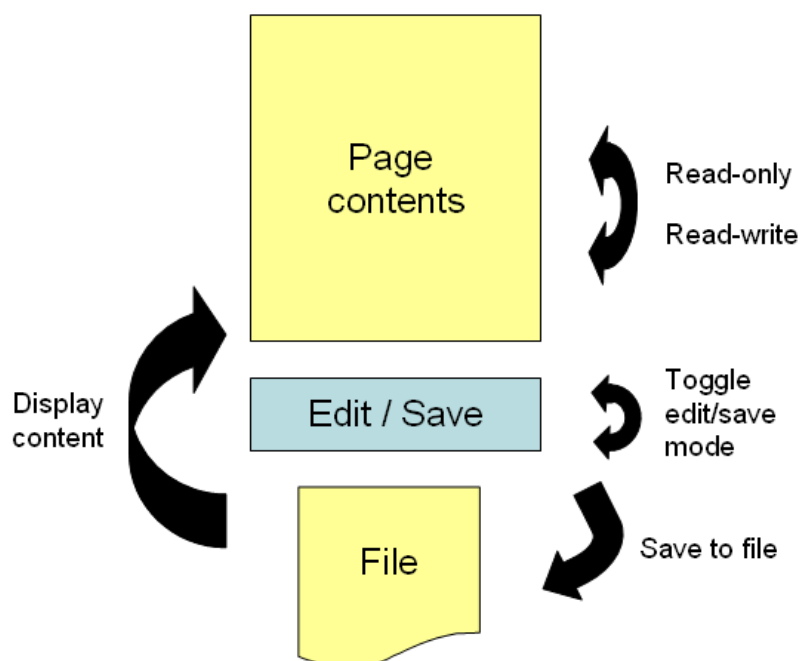
Contents

- [Introduction](#)
- [Viewing the sample application](#)
- [About the client-side WikiLite code](#)
 - [Style information](#)
 - [The body area](#)
 - [Client-side scripting](#)
 - [init\(\)](#)
 - [setEditing\(\)](#)
 - [\\$\(\)](#)
 - [editOrSave\(\)](#)
 - [checkValid\(\)](#)
 - [Server-side scripting](#)
 - [Determining which functions run where](#)
 - [Server-side functions](#)
 - [readAndFill\(\)](#)
 - [getFilePath\(\)](#)
 - [save\(contents\)](#)
- [Testing the WikiLite application](#)
 - [Editing and saving text](#)
 - [Validating the text area contents](#)
- [Complete WikiLite application code](#)

Introduction

Your Jaxer installation includes an example of a very basic Wiki web application. This simple application allows you to enable text editing, write a few text characters, save your text to a file, and reload your data into the web page when you re-enter or refresh the page.

The diagram below shows the basic flow of this application.



This application consists of a single HTML file (**WikiLite/index.html**) that contains style information and a bit of JavaScript. Open this page in Aptana Studio to follow along as the following sections walk you through the application code.

Viewing the sample application

Your Jaxer installation (whether as a standalone Jaxer Package or in Aptana Studio) includes the WikiLite application.

Ajax, meet server.

Ever wish you could use your **Ajax**, **DOM**, and **Javascript** skills on the server? Now you can!

Download Jaxer for your own server now!

Ajax everywhere. 

To access the WikiLite application from within Aptana Studio:

1. In the Samples View, expand the **Aptana Jaxer** folder.
2. Select the **WikiLite** example.
3. Click either the **Preview Sample** button  to do a quick preview of the sample, or click the **Import Sample** button  to import the sample as a project into your workspace.

To access the WikiLite application from a standalone Jaxer Package installation:

1. Navigate to your Jaxer installation folder, and double-click the **StartServers.bat** file.
2. In your web browser, navigate to the following URL: <http://localhost:8081/aptana/>
3. From the column on the left, click the **Apps and Tools** link.
4. Click the **WikiLite** link.

About the client-side WikiLite code

Before we what happens on the server with the WikiLite application, let's take a look at the client-side code, including how the web page is laid out.

Style information

The head tag of this web page contains some basic CSS styling, contained in a set of style tags:

```
01. <style>
02.     html {
03.         height: 95%;
04.         width: 99%;
05.         padding: 0;
06.         margin: 0;
07.     }
08.
09.     #contents {
10.         width: 100%;
11.         height: 95%;
12.     }
13.
14.     #editOrSave {
15.         width: 100%;
16.     }
17.
18.     .readonly {
19.         font: 8pt verdana;
20.         background-color: #f0f0ff;
21.         border: none;
22.         padding: 8px;
23.     }
24.
25.     #applicationContent {
26.         height: 90%;
27.     }
28. </style>
```

This styling formats the text area slightly differently than the rest of the page. The `.readonly` class displays the textarea with a different font and background color when the page is in read-only mode.

The body area

The applicationContent DIV in the BODY consists of a single text area and an input button:

```
1. <input type="button" id="editOrSave" onclick="editOrSave()">
2. <textarea id="contents"></textarea>
```

The textarea holds the contents of the wiki, and the input button has an onclick event handler that toggles the "edit" functionality on when the page is read-only. The button also saves the textarea contents when the page is in "edit" mode.

Note: The body tag also has an onserverload event handler. We'll discuss the functionality of this event handler later on in this article.

Client-side scripting

The head tag contains a script tag with the JavaScript for the WikiLite application. Five of the functions in this script tag perform client-side functionality:

- `init()`
- `setEditing()`
- `$()`
- `editOrSave()`
- `checkValid()`

init()

The `<tt>init()` function is called when the page loads. It sets the page to read-only mode on the page load. It also calls the `readAndFill()` function, which reads the contents of a file and displays them in the textarea on this page. We'll discuss this in more depth later on in this article.

```
01. /*
02.  * Set the initial editing state to false and read in the last saved
03.  * value of the 'wiki'
04.  */
05. function init()
06. {
07.     setEditing(false);
08.     readAndFill();
09. }
```

setEditing()

The `setEditing()` function sets the mode (read-only or read-write) of the contents area by using the `<tt>readOnly` and `className` attributes of "contents". If the "contents" are not "editable", the "contents" are then "readonly".

This function also sets the value of the `editOrSave` button. If the "contents" are read-only, the button reads "edit". If the "contents" are "editable", the button reads "save", and the function sets the focus to the "contents" area, so that a user can easily type something in the textarea .

```
01.  /*
02.  * Set the editing state of the textarea as well as the label of the action button
03.  */
04.  function setEditing(editable)
05.  {
06.      $("contents").readOnly = !editable;
07.      $("contents").className = editable ? "" : "readonly";
08.      $("editOrSave").value = editable ? "save" : "edit"; // set the button label
09.      if (editable)
10.      {
11.          $("contents").focus();
12.      }
13.  }
```

`$()`

The `$()` function is a simple helper function to get elements by ID. If you were coding this on your own, you could choose to use the `$()` function of your favorite Ajax library.

```
1.  /*
2.  * Easy access to a named element in the DOM
3.  */
4.  function $(id)
5.  {
6.      return document.getElementById(id);
7.  }
```

`editOrSave()`

The `editOrSave()` function first determines if you are already in edit mode or not. If you are in edit mode, it does a quick validation check on the contents of the text area, and if these validate, saves the contents and re-sets the page to read-only mode.

If the contents are not valid, `editOrSave()` throws an alert containing a message describing the error.

If you are not in edit mode, `editOrSave()` simply toggles off the read-only mode for the contents textarea .

```
01.  /*
02.  * Depending on the state of the editing text area, either save the contents to the
03.  * server or go into edit mode.
04.  */
05.  function editOrSave()
06.  {
07.      var isEditing = !$("contents").readOnly;
08.      if (isEditing) // try to save and get out of edit mode
09.      {
10.          try
11.          {
12.              var contents = $("contents").value;
13.              checkValid(contents);
14.              save(contents);
15.              setEditing(false);
16.          }
17.          catch (e)
18.          {
19.              alert(e);
20.          }
21.      }
22.      else // go into edit mode
23.      {
24.          setEditing(true);
25.      }
26.  }
```

`checkValid()`

The `checkValid()` function does a quick validation check on the number of characters input into "contents". If a user types more than 10000, it throws an error message saying "Too long!".

```
01.  /*
02.  * Check to make sure our wiki entry isn't too long, this is arbitrary
03.  */
04.  function checkValid(contents)
05.  {
06.      if (contents.length > 10000)
07.      {
08.          throw "Too long!";
09.      }
10.  }
```

Server-side scripting

Now that you have a basic idea of how the page is set up and how it functions, we'll examine how this page interacts with the Jaxer server. By adding server-side functionality, we're able to save the text that you type into "contents" and have it persist from one session to another.

Since we'll be running our JavaScript on the server, we need to specify this in our `script` tag. If you look at the `script` tag in our sample code, you'll see that we've added a `runat="server"` attribute:

```
1.  <script type="text/javascript" runat="server">
```

Determining which functions run where

Out of the functions described in the above section, we'll need several of them to be available both on the client and on the server. On line 54, we set the `setEditing` function to be available to be run on both:

```
setEditing.runat = "both";
```

We need this function available both on the server when we prepare the page and on the client when the user toggles editing mode on and off.

On line 60, we set the `$()` function to run at both:

```
$.runat = "both";
```

This is so that we can perform various Ajax functionality and DOM manipulation on both the front and back end of the application.

On line 93, we also set the `checkValid` function to run at both:

```
checkValid.runat = "both";
```

We need to be able to validate the contents on both the client and the server. The client performs an initial check, but the server validation catches anything not caught by the client.

Because the `editOrSave` function is only applicable to the client, we set its `runat` value to "client" only.

```
editOrSave.runat = "client";
```

Finally, remember the `init()` function that was attached to an event handler in the `body` tag? We want the page to be initialized when the server loads, so this is when we specify for `init()` to be called:

```
<body onload="init()">
```

Server-side functions

The WikiLite application has three functions that are specific to server-side functionality:

- `readAndFill()`
- `getFilePath()`
- `save(contents)`

`readAndFill()`

The `readAndFill()` function gets the contents of a file that is obtained by calling the `getFilePath` function. It checks if a file exists at the filepath, and if the file exists, gets the file contents and sets the value "contents" to those file contents. If the file does not exist, `readAndFill()` leaves "contents" blank.

```
01.  /*
02.   * Open up the file we're using to save data and read in its contents
03.   * and set that to the value of the textarea.
04.   */
05.  function readAndFill()
06.  {
07.      var filePath = getFilePath();
08.      var fileExists = Jaxer.File.exists(filePath);
09.      $("contents").value = fileExists ? Jaxer.File.read(filePath) : "";
10.  }
```

`getFilePath()`

The `getFilePath()` function returns the "contents.txt" file. (This has been hard-coded for the sake of the demo application):

```
1.  /*
2.   * Return the filename of the file that will persist our wiki contents
3.   */
4.  function getFilePath()
5.  {
6.      return "contents.txt";
7.  }
```

`save(contents)`

The `save(contents)` function normalizes newline characters for Windows and Mac and then does an additional validation check on the contents. `save` then finally writes "contents" to the file at `getFilePath`.

```
01.  /*
02.   * Do the actual saving of the text that was entered.
03.   */
04.  function save(contents)
05.  {
06.      contents = contents.replace(/\r\n/g, "\n"); // normalize newlines for Windows/IE;
07.      contents = contents.replace(/\r/g, "\n"); // normalize newlines for Mac;
08.      checkValid(contents);
09.      Jaxer.File.write(getFilePath(), contents);
10.  }
11.  // Because we want this function available to the client as well, we set its 'proxy'
12.  // value to 'true'
13.  save.proxy = true;
```

We set the proxy value of this function to "true" so that the function can be called from the client, but the contents are actually saved on the server.

```
save.proxy = true;
```

Testing the WikiLite application

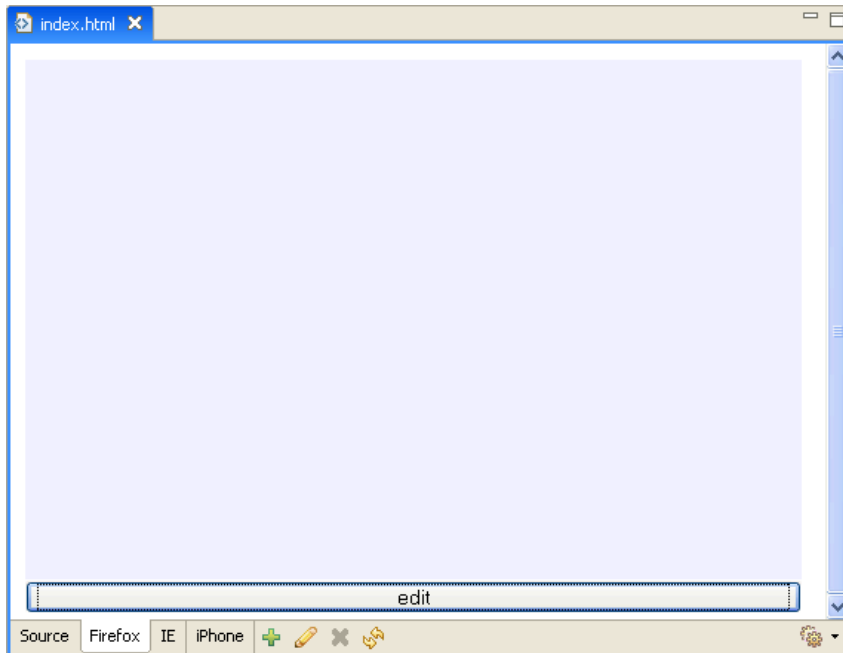
This section walks you through a few quick tests to see how the functionality of the WikiLite application works. You'll add some text to the page, save that text to a file on the server, and perform a validation check both on the client and then on the server.

Editing and saving text

To test the functionality of the WikiLite application:

1. With the **index.html** file for the WikiLite application open, click the Firefox tab to preview the application in **Firefox**. The preview should look like the screen below.

When the page loads, the `onserverload` event triggers the `init()` function, which in turn calls the `setEditing` function to set the text area to read-only. The `init()` function also calls `readAndFill` which reads the contents of a remote file (if it exists) and fills the text area with those contents .



2. Click the **edit** button.

WikiLite enables the text area for typing, which switches from a blue background to a white background. It also changes the name of the button from "edit" to "save". These events happen as a result of calling the `editOrSave` function.

3. In the text area, type a few characters, and click the **save** button.

Clicking **save** calls `editOrSave` again and toggles edit mode off. It also calls the `save` function, which normalizes whitespace, validates the text contents, and writes the contents to a file on the Jaxer server.

Validating the text area contents

The `checkValid` function checks whether the text entered into "contents" meets its validity criteria. As currently written, the text must simply be less than 10000 characters. This function is run on the client when it is called by the `editOrSave` function, and it is also run on the server when it is called by the `save` function. To test this function yourself, you can make a couple of minor changes to the code.

To test validity checking on the client:

1. Go to the `checkValid` function in **index.html**.
2. Change the length limit from 10000 to 10 (shown below):

```
01.  /*
02.   * Check to make sure our wiki entry isn't too long, this is arbitrary
03.   */
04.  function checkValid(contents)
05.  {
06.      if (contents.length > 10)
07.      {
08.          throw "Too long!";
09.      }
10.  }
```

3. Test the client-side validation:
 1. Click the Firefox preview tab to preview the page in Firefox.
 2. Click the **edit** button to toggle on edit mode.
 3. In the text area, type over 10 characters of text.
 4. Click the **save** button.

The WikiLite app displays a "Too long!" error telling you that the text that you entered is too long.

To test validity on the server:

1. Leave the `checkValid` function with the modification that you made above.
2. Comment out the validity check on the client-side:
 1. Go to the `editOrSave` function (line 63).
 2. Comment out line 13 (as shown below), where the `checkValid` function is called (shown below):

```
01.  /*
02.   * Depending on the state of the editing text area, either save the contents to the
03.   * server or go into edit mode.
04.   */
05.  function editOrSave()
06.  {
```

```

007.     var isEditing = !$("#contents").readOnly;
008.     if (isEditing) // try to save and get out of edit mode
009.     {
010.         try
011.         {
012.             var contents = $("#contents").value;
013.             checkValid(contents);
014.             save(contents);
015.             setEditing(false);
016.         }
017.         catch (e)
018.         {
019.             alert(e);
020.         }
021.     }
022.     else // go into edit mode
023.     {
024.         setEditing(true);
025.     }
026. }

```

3. Test the client-side validation:

1. Click the Firefox preview tab to preview the page in Firefox.
2. Click the **edit** button to toggle on edit mode.
3. In the text area, type over 10 characters of text.
4. Click the **save** button.

The WikiLite app displays a "Too long!" server error telling you that the text that you entered is too long.

Complete WikiLite application code

The following code sample is the entire page of code for the WikiLite application:

```

001. <html>
002. <head>
003.     <title>WikiLite</title>
004.     <link rel="stylesheet" type="text/css" href="http://jaxer.org/tutorials/css/index.css">
005.
006.     <style>
007.         html {
008.             height: 95%;
009.             width: 99%;
010.             padding: 0;
011.             margin: 0;
012.         }
013.
014.         #contents {
015.             width: 100%;
016.             height: 95%;
017.         }
018.
019.         #editOrSave {
020.             width: 100%;
021.         }
022.
023.         .readonly {
024.             font: 8pt verdana;
025.             background-color: #f0f0ff;
026.             border: none;
027.             padding: 8px;
028.         }
029.
030.         #applicationContent {
031.             height: 90%;
032.         }
033.     </style>
034.
035.     <!-- This script block is a standard script block and runs on the browser only -->
036.     <script type="text/javascript">
037.         /*
038.          * Depending on the state of the editing text area, either save the contents to the
039.          * server or go into edit mode.
040.          */
041.         function editOrSave()
042.         {
043.             var isEditing = !$("#contents").readOnly;
044.             if (isEditing) // try to save and get out of edit mode
045.             {
046.                 try
047.                 {
048.                     var contents = $("#contents").value;
049.                     checkValid(contents);
050.                     save(contents);
051.                     setEditing(false);
052.                 }
053.                 catch (e)
054.                 {
055.                     alert(e);
056.                 }
057.             }
058.             else // go into edit mode
059.             {
060.                 setEditing(true);
061.             }
062.         }
063.     </script>
064.
065.     <!-- This script block will execute only on the server because of the
066.          runat="server" attributes. -->
067.     <script type="text/javascript" runat="server">
068.
069.         /*
070.          * Set the initial editing state to false and read in the last saved
071.          * value of the 'wiki'
072.          */
073.         function init()
074.         {

```

```

075.         setEditing(false);
076.         readAndFill();
077.     }
078.
079.     /*
080.     * Open up the file we're using to save data and read in its contents
081.     * and set that to the value of the textarea.
082.     */
083.     function readAndFill()
084.     {
085.         var filePath = getFilePath();
086.         var fileExists = Jaxer.File.exists(filePath);
087.         $("contents").value = fileExists ? Jaxer.File.read(filePath) : "";
088.     }
089.
090.     /*
091.     * Return the filename of the file that will persist our wiki contents
092.     */
093.     function getFilePath()
094.     {
095.         return "contents.txt";
096.     }
097.
098.     /*
099.     * Do the actual saving of the text that was entered.
100.     */
101.     function save(contents)
102.     {
103.         contents = contents.replace(/\r\n/g, "\n"); // normalize newlines for Windows/IE;
104.         contents = contents.replace(/\r/g, "\n"); // normalize newlines for Mac;
105.         checkValid(contents);
106.         Jaxer.File.write(getFilePath(), contents);
107.     }
108.     // Because we want this function available to the client as well, we set its 'proxy'
109.     // value to 'true'
110.     save.proxy = true;
111.
112. </script>
113.
114. <!-- This script block will be available on both client and server since the
115.      runat attribute is set to 'both'. -->
116. <script type="text/javascript" runat="both">
117.     /*
118.     * Easy access to a named element in the DOM
119.     */
120.     function $(id)
121.     {
122.         return document.getElementById(id);
123.     }
124.
125.     /*
126.     * Set the editing state of the textarea as well as the label of the action button
127.     */
128.     function setEditing(editable)
129.     {
130.         $("contents").readOnly = !editable;
131.         $("contents").className = editable ? "" : "readonly";
132.         $("editOrSave").value = editable ? "save" : "edit"; // set the button label
133.         if (editable)
134.         {
135.             $("contents").focus();
136.         }
137.     }
138.
139.     /*
140.     * Check to make sure our wiki entry isn't too long, this is arbitrary
141.     */
142.     function checkValid(contents)
143.     {
144.         if (contents.length > 10000)
145.         {
146.             throw "Too long!";
147.         }
148.     }
149. </script>
150.
151. </head>
152.
153. <!-- We set a new attribute on the body tag: 'onserverload'. It works just like 'onload', but
154.      on the server-side. When Jaxer is done loading the page, the function specified will
155.      be executed. You can have both onserverload and onload, for server and client,
156.      respectively. -->
157. <body onserverload="init()">
158.     <script type="text/javascript" src="http://jaxer.org/tutorials/lib/wz_tooltip.js"></script>
159.     <div id="sampleDescription">
160.         An extremely lightweight single-page Wiki, just an editable box that persists to a file
161.     </div>
162.     <div id="sampleSource">
163.     <li><a href="http://jaxer.org/aptana/tools/sourceViewer/index.html?filename=../../samples/wikilite/index.html" target="_blank">Main
164.     <page</a></li>
165.     </div>
166.
167.     <div id="sampleHeader">
168.         <div class="sampleTitle">
169.              WikiLite
170.         </div>
171.         <div id="rightFloat">
172.             
174.             <span id="sourceButton"></span>
176.         </div>
177.     </div>
178.
179.     <div id="applicationContent">
180.         <input type="button" id="editOrSave" onclick="editOrSave()">
181.         <textarea id="contents"></textarea>
182.     </div>

```

```
180.
181.     <div id="sampleFooter">
182.     </div>
183.     <script runat="server">
184.         if (Jaxer.System.executableFolder.match('com.apтана.іde.framework.jaxer')) {
185.             document.getElementById('sampleSource').innerHTML="";
186.             document.getElementById('sourceButton').innerHTML="";
187.         }
188.     </script>
189. </body>
190.
191. </html>
```